Data Warehouse: Concepts

Khalid Belhajjame

Data Analysis

- The importance of **data analysis** has been steadily increasing in all sectors to **improve decision-making processes** and maintain competitive advantage.
- **Traditional database systems** do not satisfy the requirements of data analysis.
  - their design should prevent update anomalies
  - ➢ and thus they **are highly normalized**
  - ➢ and therefore they **perform poorly when executing complex join queries** or aggregate large volumes of data.
  - they **do not include historical data**.
- This led to the notion of **data warehouses**, which are (usually) **large repositories** that consolidate data **from different sources**, are **updated off-line**, and follow the **multidimensional data model**.

Data Warehouse

A data warehouse is a repository of integrated data obtained from several sources for the specific purpose of multidimensional data analysis.

- **Subject oriented**: data warehouses focus on the analytical needs of different areas of an organization.
  - For example, in the case of a retail company, the analysis may focus on product sales or inventory management.
- **Integrated**: data is obtained from several sources must be joined together
  - Issues: differences in data definition and content, such as differences in data format and data codification, synonyms (fields with different names but the same data) and other issues

Data Warehouse

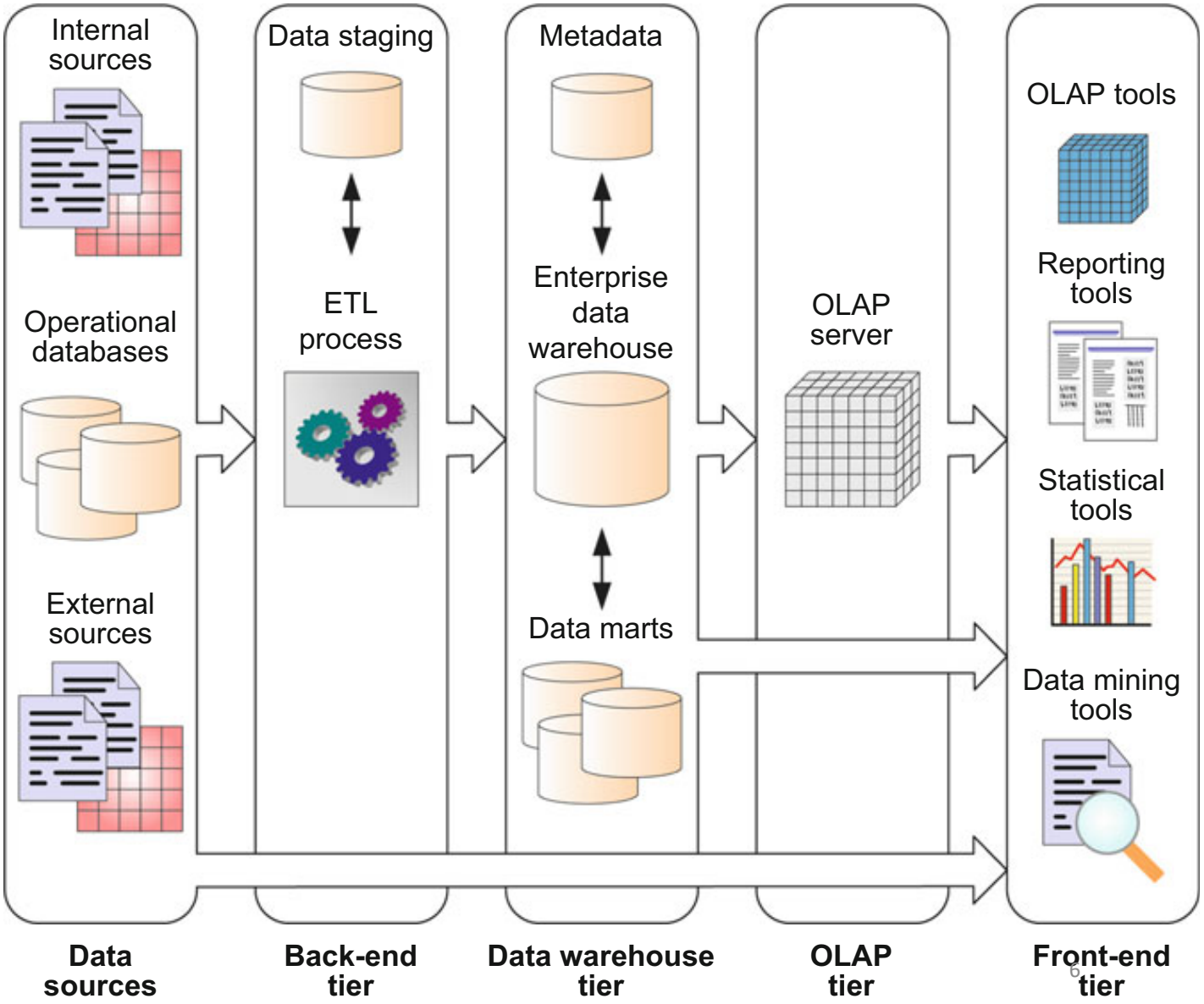
A data warehouse is a repository of integrated data obtained from several sources for the specific purpose of multidimensional data analysis.

- **Non-volatile:** durability of data is ensured by disallowing data modification and removal.
  - A data warehouse gathers data encompassing several years, typically 5–10 years or beyond.
- **Time varying**: retaining different values for the same information, as well as the time when changes to these values occurred.
  - For example, a data warehouse in a bank might store information about the average monthly balance of clients' accounts for a period covering several years.

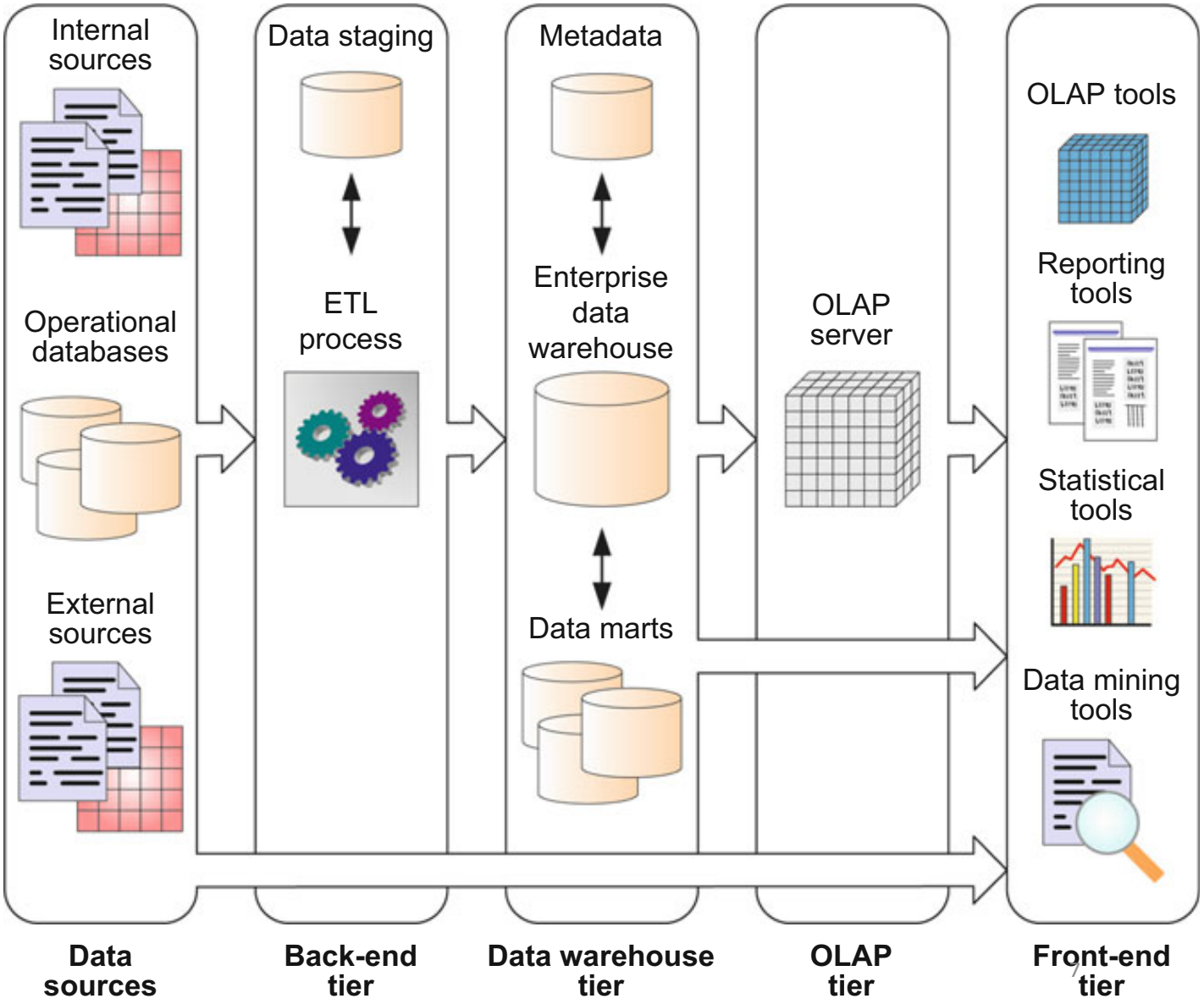
Data Warehouse

- A data warehouse is aimed at analyzing the data of an entire organization.
- It is often the case that particular departments or divisions of an organization only require a portion of the organizational data warehouse specialized for their needs.
- For example, a sales department may only need sales data, while a human resources department may need demographic data and data about the employees.
- These departmental data warehouses are called **data marts**.
- These data marts are not necessarily private to a department; they may be shared with other interested parts of the organization.

Data Warehouse Architecture



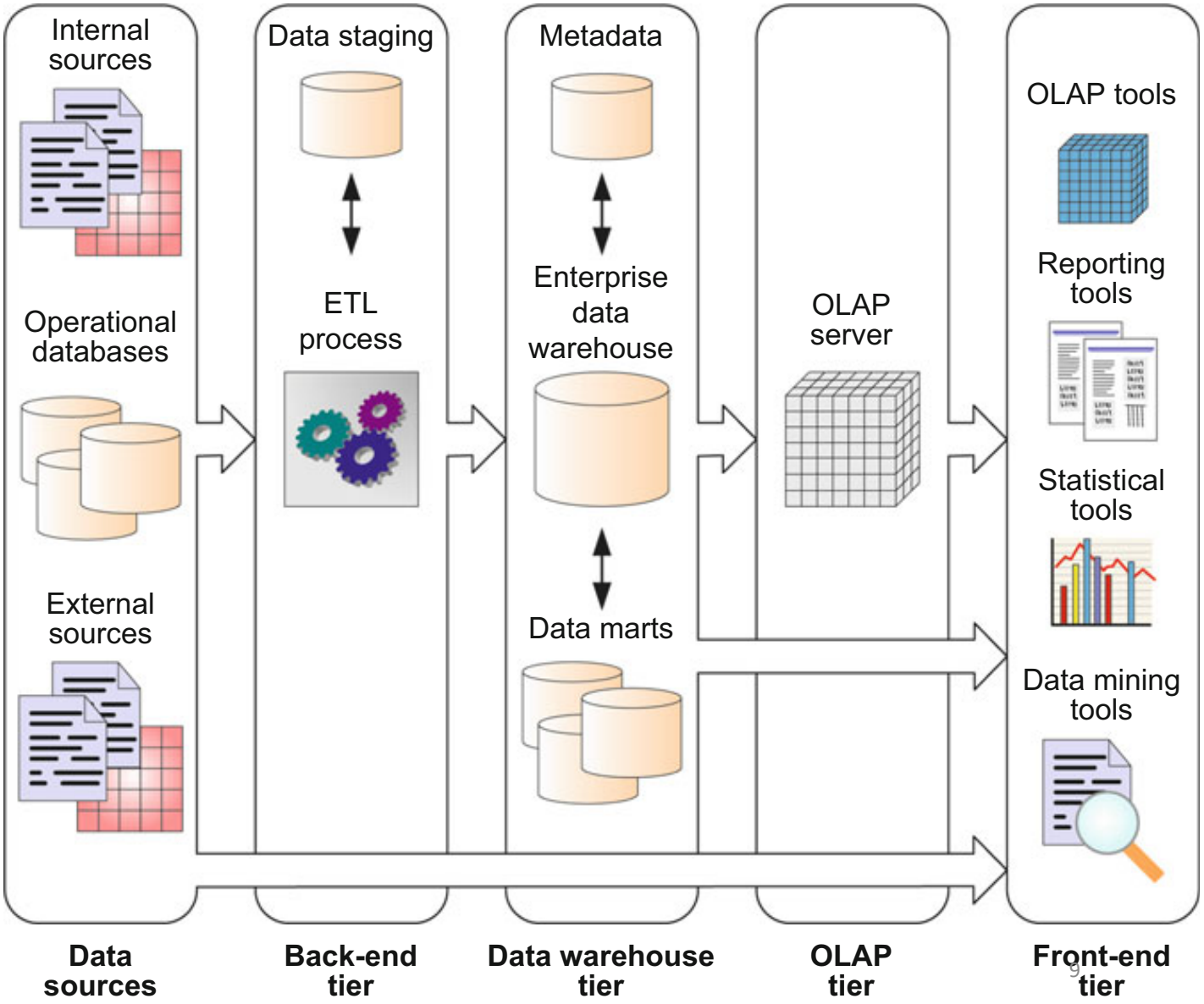
The **back-end tier** is composed of extraction, transformation, and loading (**ETL**) tools, used to feed data into the data warehouse from operational databases and other data sources.



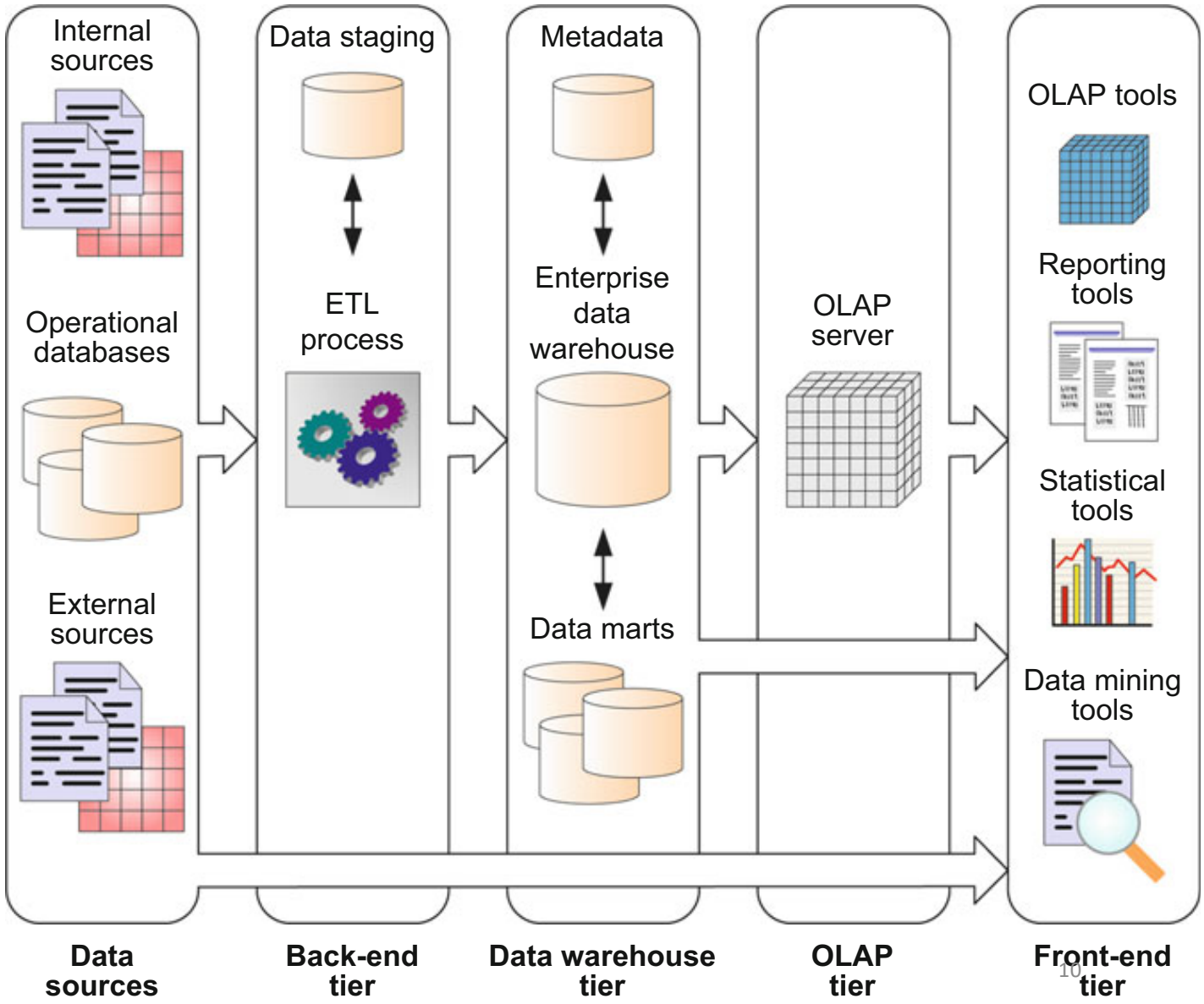
ETL

- **Extraction** gathers data from multiple, heterogeneous data sources. These sources may be operational databases but may also be files in various formats; they may be internal to the organization or external to it.
- **Transformation** modifies the data from the format of the data sources to the warehouse format. This includes several aspects:
  - cleaning, which removes errors and inconsistencies in the data and converts it into a standardized format;
  - integration, which reconciles data from different data sources, both at the schema and at the data level;
  - aggregation, which summarizes the data obtained from data sources according to the level of detail, or granularity, of the data warehouse.
- **Loading** feeds the data warehouse with the transformed data.
  - This also includes refreshing the data warehouse, that is, propagating updates from the data sources to the data warehouse at a specified frequency in order to provide up-to-date data for the decision-making process.

The data warehouse tier is composed of an enterprise data warehouse and/or several data marts and a metadata repository storing information about the data warehouse and its contents.

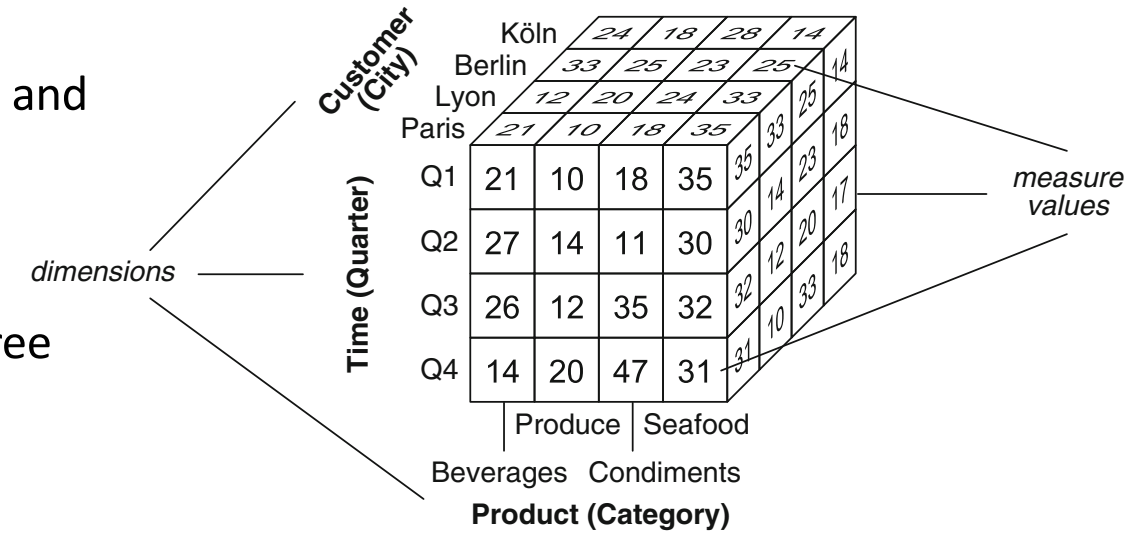


The OLAP tier is composed of an OLAP server, which provides a multidimensional view of the data, regardless of the actual way in which data are stored in the underlying system.



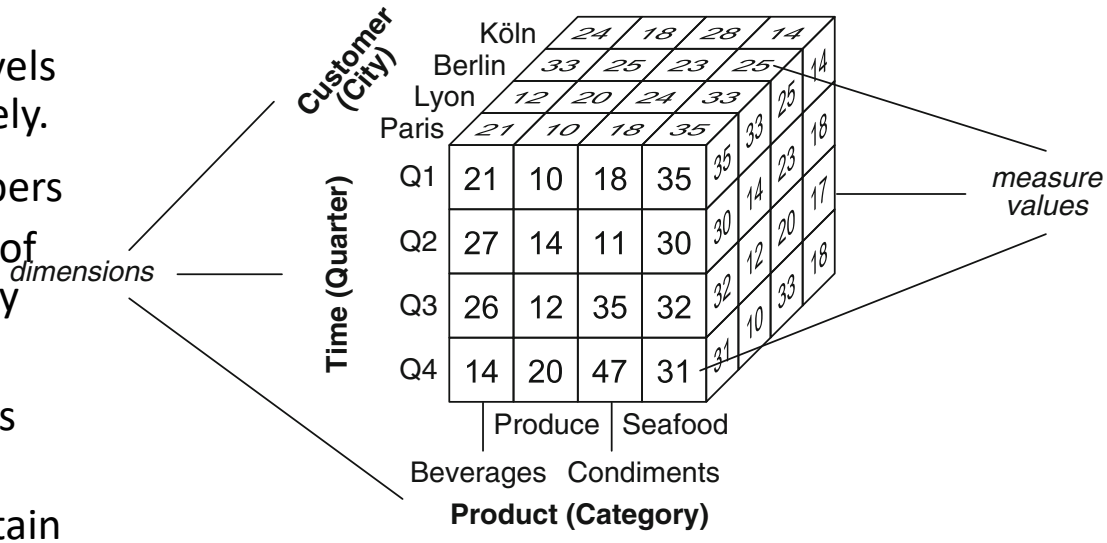
Multidimentional Model

- Multidimensional model views data in an n-dimensional space, called a **data cube** or a hypercube.
- A data cube is defined by **dimensions** and **facts**.
- Dimensions are **perspectives used to analyze the data**
  - E.g., the cube on the right has three dimensions: Product, Time and Customer



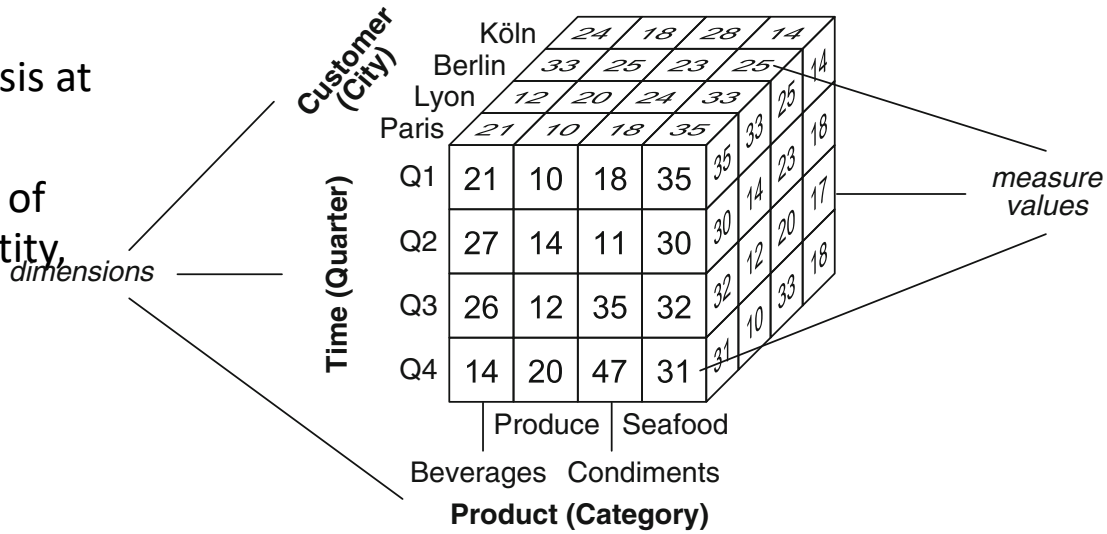
Multidimentional Model

- A **dimension level** represents the granularity at which measures are represented for each dimension.
  - Sales figures are aggregated to the levels Category, Quarter, and City, respectively.
- **Instances of a dimension** are called members
  - Seafood and Beverages are members of the Product dimension at the Category level.
- Dimensions also have associated attributes describing them.
  - E.g, the Product dimension could contain attributes such as ProductNumber and UnitPrice, which are not shown in the figure.



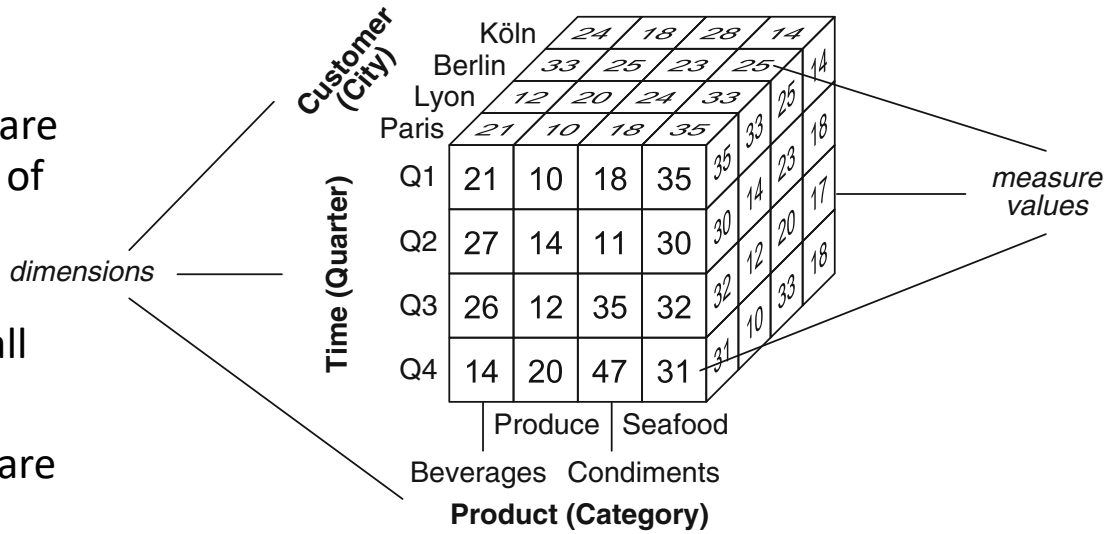
Multidimentional Model

- The **cells** of a data cube, or facts, have associated numeric values called **measures**.
- These measures are used to evaluate quantitatively various aspects of the analysis at hand.
- For example, each number shown in a cell of the data cube represents a measure Quantity, indicating the number of units sold (in thousands) by category, quarter, and customer's city.
- A data cube typically contains **several measures**.
  - For example, another measure, not shown in the figure, could be Amount, indicating the total sales amount.



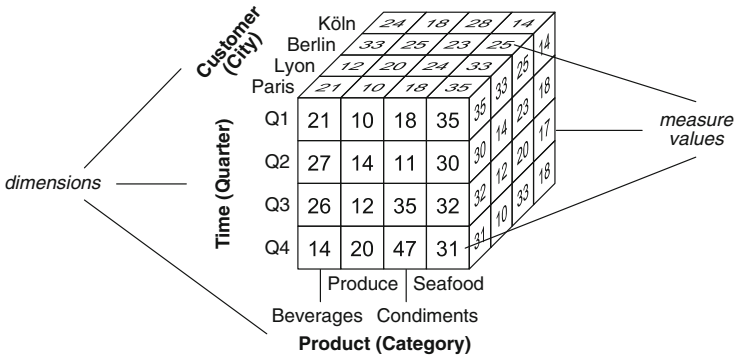
Multidimentional Model

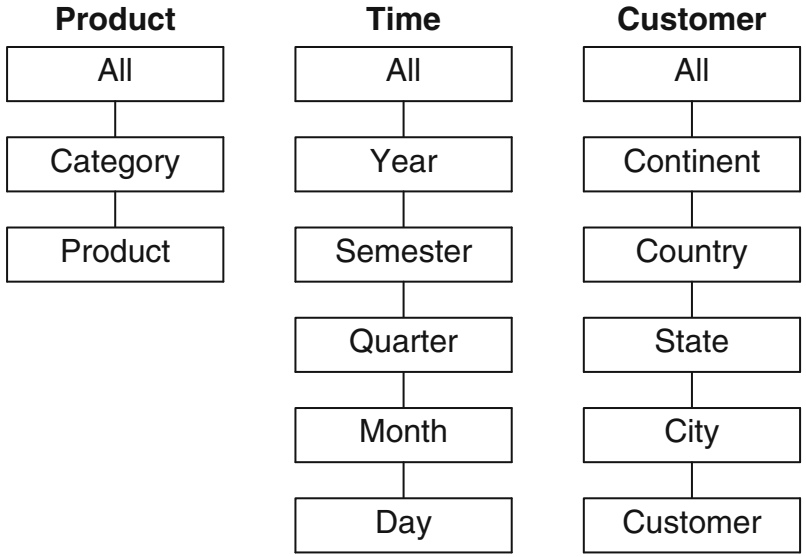
- A data cube may be sparse or dense depending on whether it has measures associated with each combination of dimension values.
- E.g, this depends on whether all products are bought by all customers during the period of time considered.
- For example, not all customers may have ordered products of all categories during all quarters of the year.
- Actually, in real-world applications, cubes are typically sparse.



Hierarchies

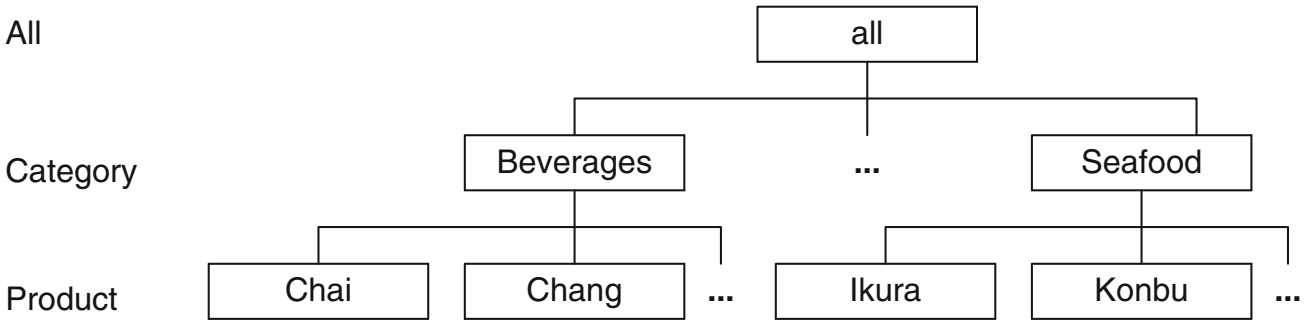
- The **granularity** of a data cube is determined by the **combination of the levels** corresponding to each axis of the cube.
  - the dimension levels are indicated between parentheses
- To extract strategic knowledge from a cube, it is necessary to view its data at several levels of detail.
  - an analyst may want to see the sales figures at a finer granularity, such as at the month level, or at a coarser granularity, such as at the customer's country level.
- **Hierarchies** allow this possibility by defining a sequence of mappings relating lower-level, detailed concepts to higher-level, more general concepts.
- The hierarchical structure of a dimension is called the **dimension schema**.





Hierarchies

- A **dimension instance** comprises the members at all levels in a dimension.
- The figure below shows an example of the Product dimension at the instance level.



- Each product at the lowest level of the hierarchy can be mapped to a corresponding category.
- All categories are grouped under a member called all, which is the only member of the distinguished level All.
- This member is used for obtaining the aggregation of measures for the whole hierarchy, that is, for obtaining the total sales for all products.

Measures

- Each measure in a cube is associated with an aggregation function that combines several measure values into a single one.
- Aggregation of measures takes place when one changes the level of detail at which data in a cube are visualized.
- E.g, if we use the Customer hierarchy for changing the granularity of the data cube from City to Country, then the sales figures for all customers in the same country will be aggregated using the SUM operation.
- Total sales figures will result in a cube containing one cell with the total sum of the quantities of all products

Measures

- **Summarizability** refers to the correct aggregation of cube measures along dimension hierarchies, in order to obtain consistent aggregation results.
- To ensure summarizability, the following properties need to be satisfied
  - **Disjointness of instances**: a product cannot belong to two categories.
    - If this were the case, each product sales would be counted twice, one for each category.
  - **Completeness**: All instances must be included in the hierarchy and each instance must be related to one parent in the next level.
  - **Correctness**: refers to the correct use of the aggregation functions. Measures can be of various types, and this determines the kind of aggregation function to apply.

Measures: Types of Aggregations

Measures can be classified as follows:

- Additive measures can be meaningfully summarized along all the dimensions, using addition. These are the most common type of measures.
- Semiadditive measures can be meaningfully summarized using addition along some, but not all, dimensions.
  - A typical example is that of inventory quantities, which cannot be meaningfully aggregated in the Time dimension, for instance, by adding the inventory quantities for two different quarters.
- Nonadditive measures cannot be meaningfully summarized using addition across any dimension.
  - Typical examples are item price, cost per unit, and exchange rate.

Measures: Types of Aggregations

- to define a measure, it is necessary to determine the aggregation functions that will be used in the various dimensions.
- This is particularly important in the case of semiadditive and nonadditive measures.
- For example, *how can a measure representing inventory quantities be aggregated along the Time dimension?*
- It can be aggregated computing the average along the Time dimension and computing the sum along other dimensions.
- Averaging can also be used for aggregating nonadditive measures such as item price or exchange rate.
- However, depending on the semantics of the application, other functions such as the minimum, maximum, or count could be used instead.

Optimization techniques to compute the measures

- To allow users to interactively explore the cube data at different granularities, optimization techniques based on aggregate precomputation are used.
- To avoid computing the whole aggregation from scratch each time the data warehouse is queried, OLAP tools implement incremental aggregation mechanisms.
- However, incremental aggregation computation is not always possible, since this depends on the kind of aggregation function used.
- This leads to the following classification of measures ...

Types of Measures

1. Distributive measures are defined by an aggregation function that can be computed in a distributed way.

- Suppose that the data are partitioned into n sets and that the aggregate function is applied to each set, giving n aggregated values.
- The function is distributive if the result of applying it to the whole data set is the same as the result of applying a function (not necessarily the same) to the n aggregated values.
- The usual aggregation functions such as the count, sum, minimum, and maximum are distributive.
- However, the distinct count function is not.
  - What is the distinct count of the set of measure values {3, 3, 4, 5, 8, 4, 7, 3, 8}
    - Result = 5
  - Suppose the above set is partitioned into the subsets {3, 3, 4}, {5, 8, 4}, and {7, 3, 8}, what is the result of summing up the result of the distinct count function applied to each subset
    - Result = 8
  - The two results are different => the measure is not distributive

Types of Measures

2. Algebraic measures are defined by an aggregation function that can be expressed as a scalar function of distributive ones.

- A typical example of an algebraic aggregation function is the average
- What is the distinct count of the set of measure values {3, 3, 4, 5, 8, 4, 7, 3, 8}?
  - (3+3+4+5+8+4+7+3+8) / 9 = 45/9 = 5
- Suppose the above set is partitioned into the subsets {3, 3, 4}, {5, 8, 4}, and {7, 3, 8}. How can we compute the average in a distributed manner in this case?
  - It can be computed by dividing the sum by the count, the latter two functions being distributive.
  - {3, 3, 4} -> (10, 3) // the pair represents the sum and count of the subset
  - {5, 8, 4} -> (16,3)
  - {7, 3, 8} -> (18,3)

  => Result = (10 +16 + 18)/ (3+3+3) = 45/9 = 5

Types of Measures

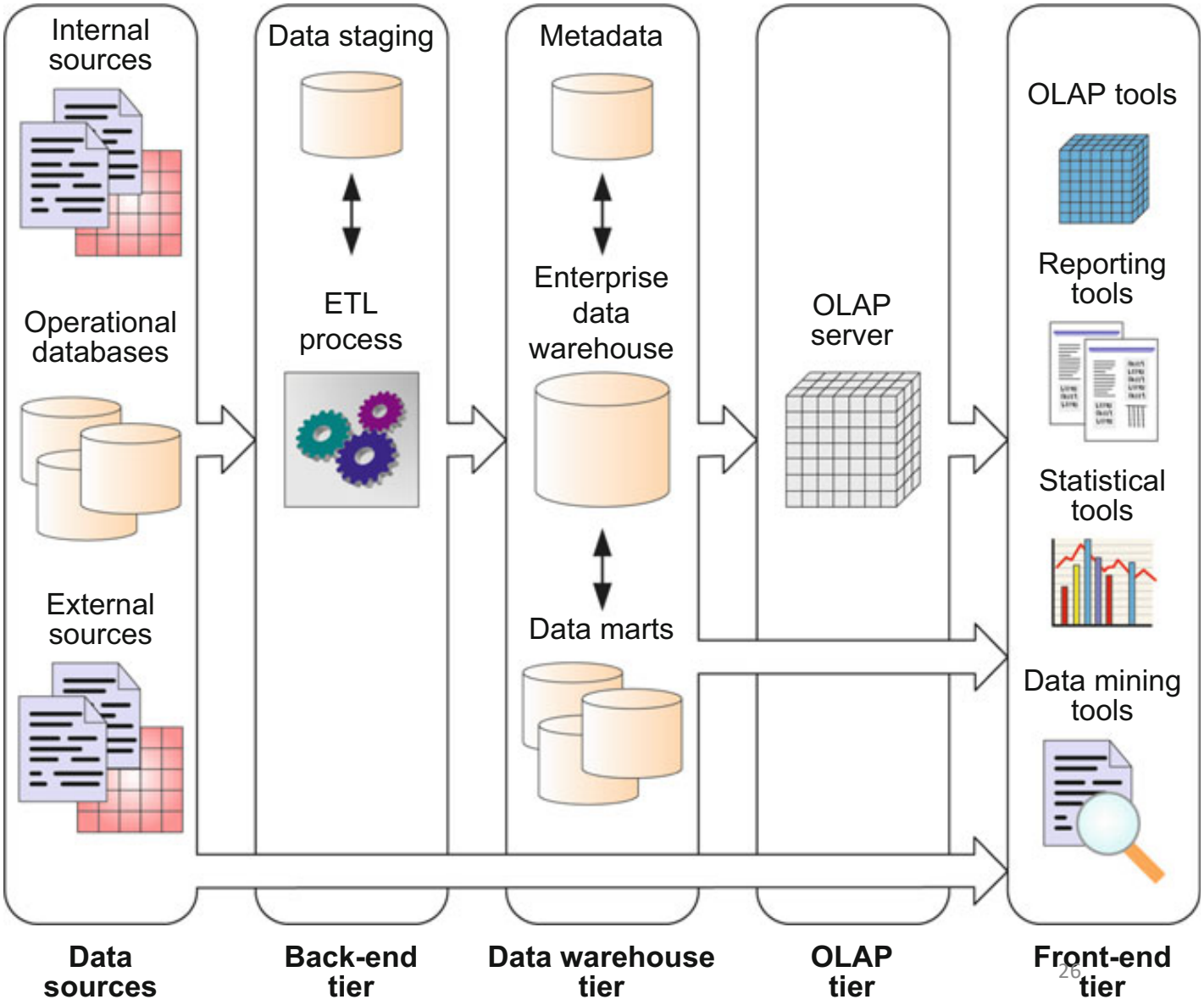
3. Holistic measures are measures that cannot be computed from other subaggregates.

- Typical examples include
  - the median: the value separating the higher half from the lower half of a data sample
    - The median of 1, 3, 3, 6, 7, 8, 9 is 6.
  - the mode: the value that occurs most often.
    - The median of 1, 3, 3, 6, 7, 8, 9 is 3.
- Holistic measures are expensive to compute, especially when data are modified, since they must be computed from scratch.

OLAP Tier

- Most database products provide OLAP extensions and related tools allowing the construction and querying of cubes, as well as navigation, analysis, and reporting.
- However, there is not yet a standardized language for defining and manipulating data cubes, and the underlying technology differs between the available systems.
- **MDX** (MultiDimensional eXpressions) is a query language for OLAP databases. As it is supported by a number of OLAP vendors, MDX became a de facto standard for querying OLAP systems.
- The SQL standard has also been extended for providing analytical capabilities; this extension is referred to as **SQL/OLAP**.

The front-end tier is used for data analysis and visualization.



Front-End Tier

The front-end tier contains client tools:

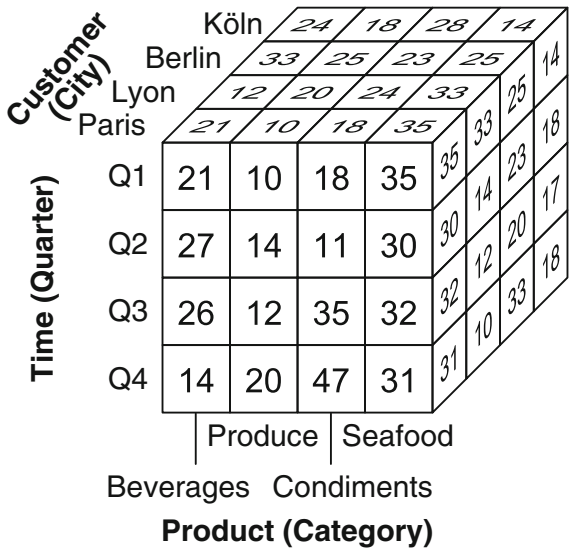
- **OLAP tools** allow interactive exploration and manipulation of the warehouse data.
- **Reporting tools** enable the production, delivery, and management of reports, which can be paper-based reports or interactive, web-based reports.
- **Statistical tools** are used to analyze and visualize the cube data using statistical methods.
- **Data mining** tools allow users to analyze data in order to discover valuable knowledge such as patterns and trends; they also allow predictions to be made on the basis of current data.

OLAP Operations

- A fundamental characteristic of the multidimensional model is that it allows one to view data from **multiple perspectives and at several levels of detail**.
- The OLAP (Online analytical processing) operations allow these perspectives and levels of detail to be materialized by exploiting the dimensions and their hierarchies.
- Thereby, they provide an interactive data analysis environment.

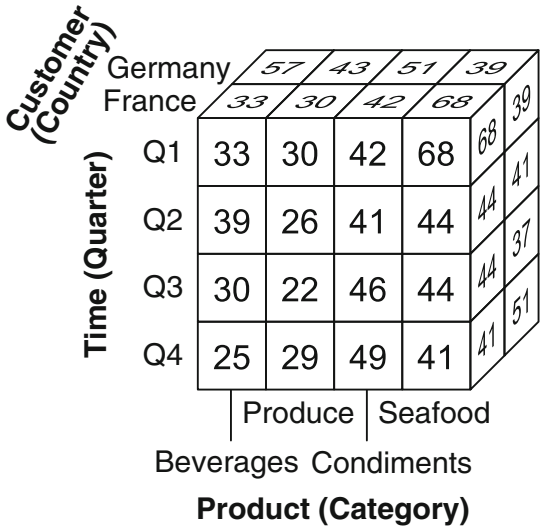
Example

contains four values in the Customer dimension,
one for each city



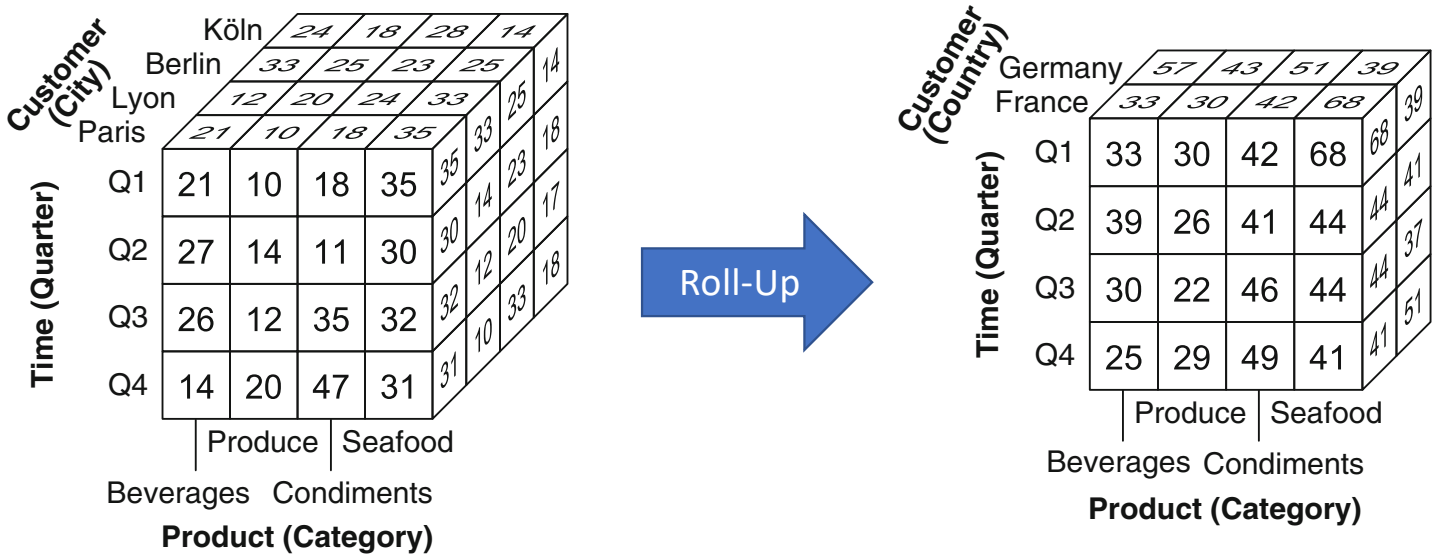
Roll-Up

contains two values,
each one corresponding to one country



The user first wants to compute the sales quantities by country.

For this, she applies a **roll-up operation** to the Country level along the Customer dimension.



- The roll-up operation aggregates measures along a dimension hierarchy to obtain measures at a coarser granularity.
- The syntax for the roll-up operation is:

$$\text{ROLLUP}(\text{CubeName}, (\text{Dimension} \rightarrow \text{Level})^*, \text{AggFunction}(\text{Measure})^*)$$

- where Dimension → Level indicates to which level in a dimension the roll-up is to be performed and function AggFunction is applied to summarize the measure.
- In the example given in the figure, we applied the operation:

$$\text{ROLLUP}(\text{Sales}, \text{Customer} \rightarrow \text{Country}, \text{SUM}(\text{Quantity}))$$

Roll-Up

- When querying a cube, a usual operation is to roll up a few dimensions to particular levels and to remove the other dimensions through a roll-up to the All level.
- In a cube with n dimensions, this can be obtained by applying n successive ROLLUP operations.
- The ROLLUP* operation provides a shorthand notation for this sequence of operations. The syntax is as follows:

  ROLLUP*(CubeName, [(Dimension → Level)*], AggFunction(Measure)*)

- For example, the total quantity by quarter can be obtained as follows:

  ROLLUP*(Sales, Time → Quarter, SUM(Quantity))

- which performs a roll-up along the Time dimension to the Quarter level and the other dimensions (in this case Customer and Product) to the All level.
- On the other hand, if the dimensions are not specified as in

  ROLLUP*(Sales, SUM(Quantity))

- all the dimensions of the cube will be rolled up to the All level, yielding a single cell containing the overall sum of the Quantity measure.

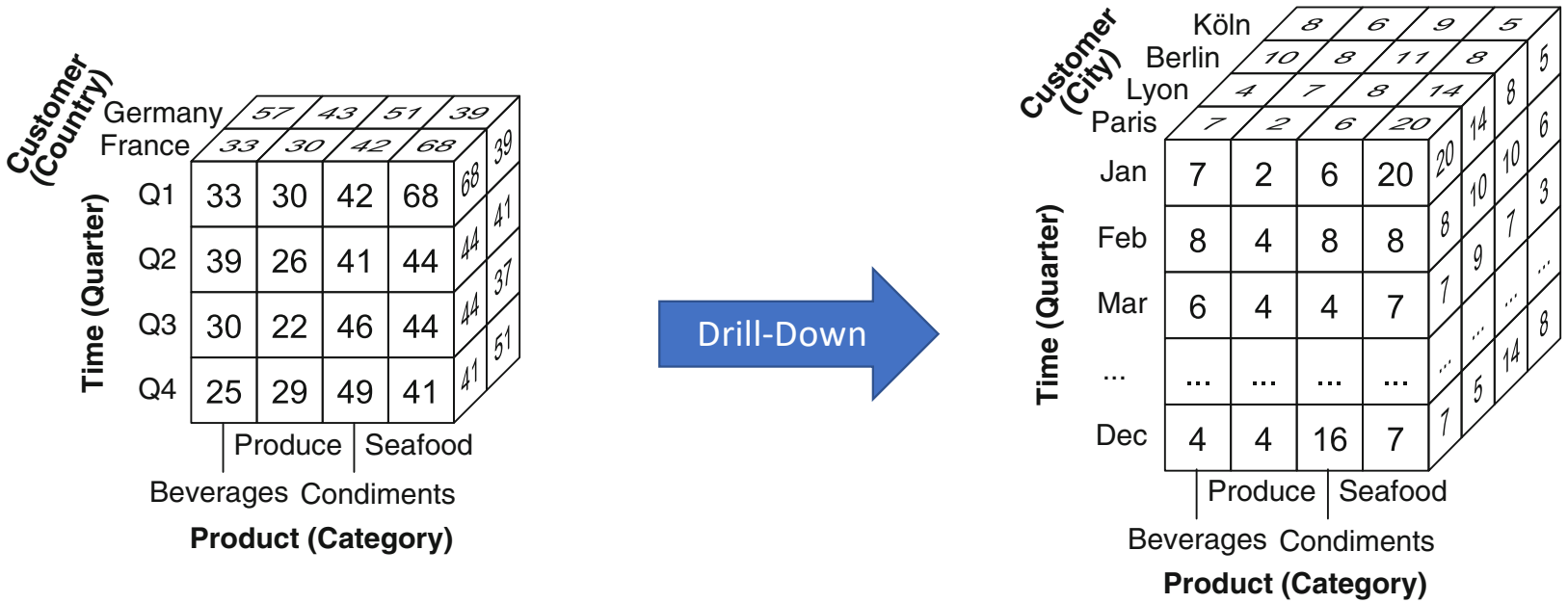
Roll-Up

- A usual need when applying a roll-up operation is to count the number of members in one of the dimensions removed from the cube. For example, the following query obtains the number of distinct products sold by quarter:

ROLLUP*(Sales, Time → Quarter, COUNT(Product) AS ProdCount)

- In this case, a new measure ProdCount will be added to the cube.

Example

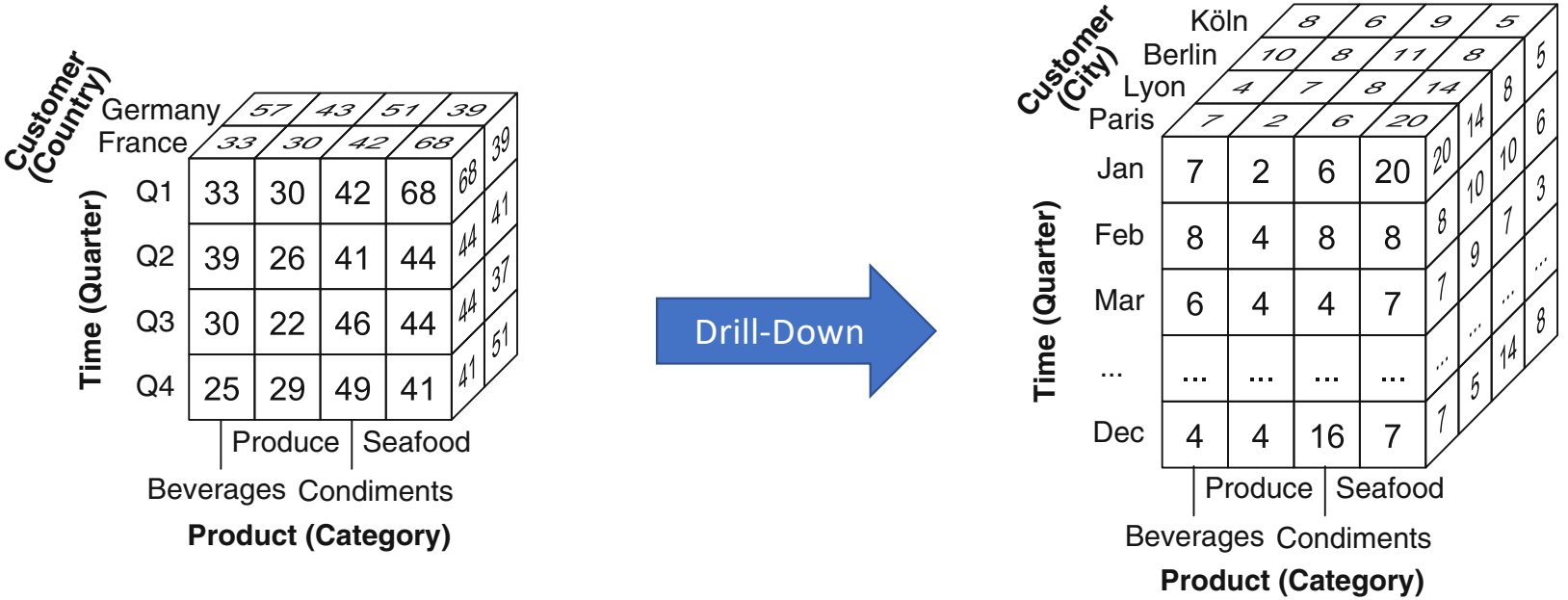


Our user then notices that sales of the category Seafood in France are significantly higher in the first quarter compared to the other ones

Thus, she first takes the cube back to the City aggregation level and then applies a drill-down along the Time dimension to the Month level to find out whether this high value occurred during a particular month.

She discovers that, for some reason yet unknown, sales in January soared both in Paris and in Lyon.

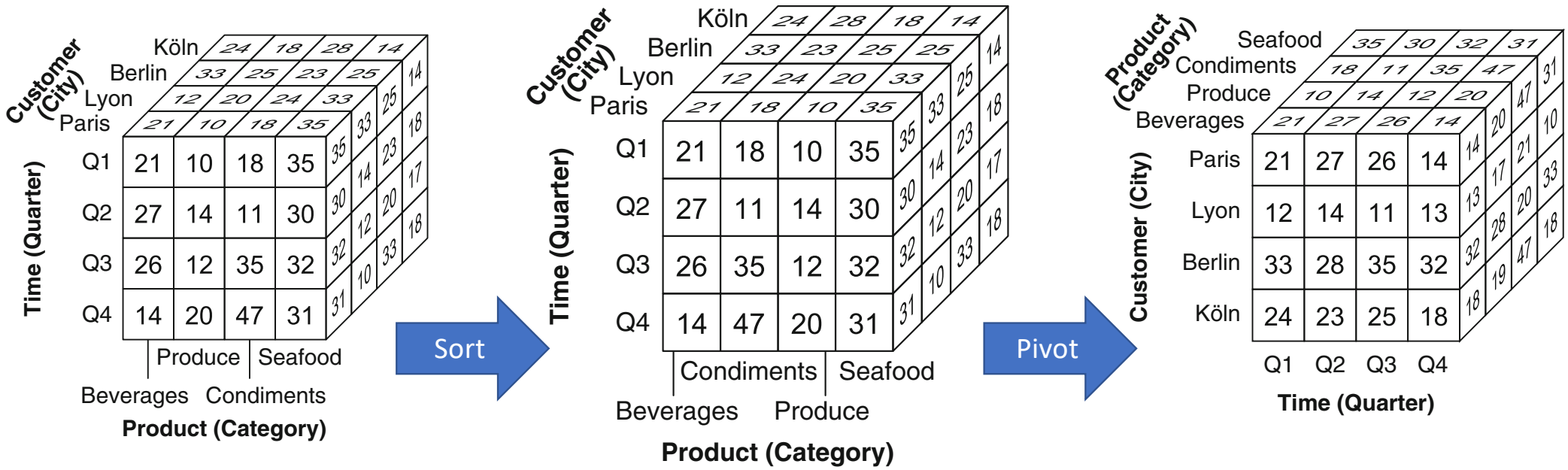
Drill-Down



- The drill-down operation performs the inverse of the roll-up operation, that is, it goes from a more general level to a more detailed level in a hierarchy.
- The syntax of the drill-down operation is as follows:

  DRILLDOWN(CubeName, (Dimension → Level)*)
- where Dimension → Level indicates to which level in a dimension we want to drill down to.
- In the above example, we applied the operation

  DRILLDOWN(Sales, Time → Month)



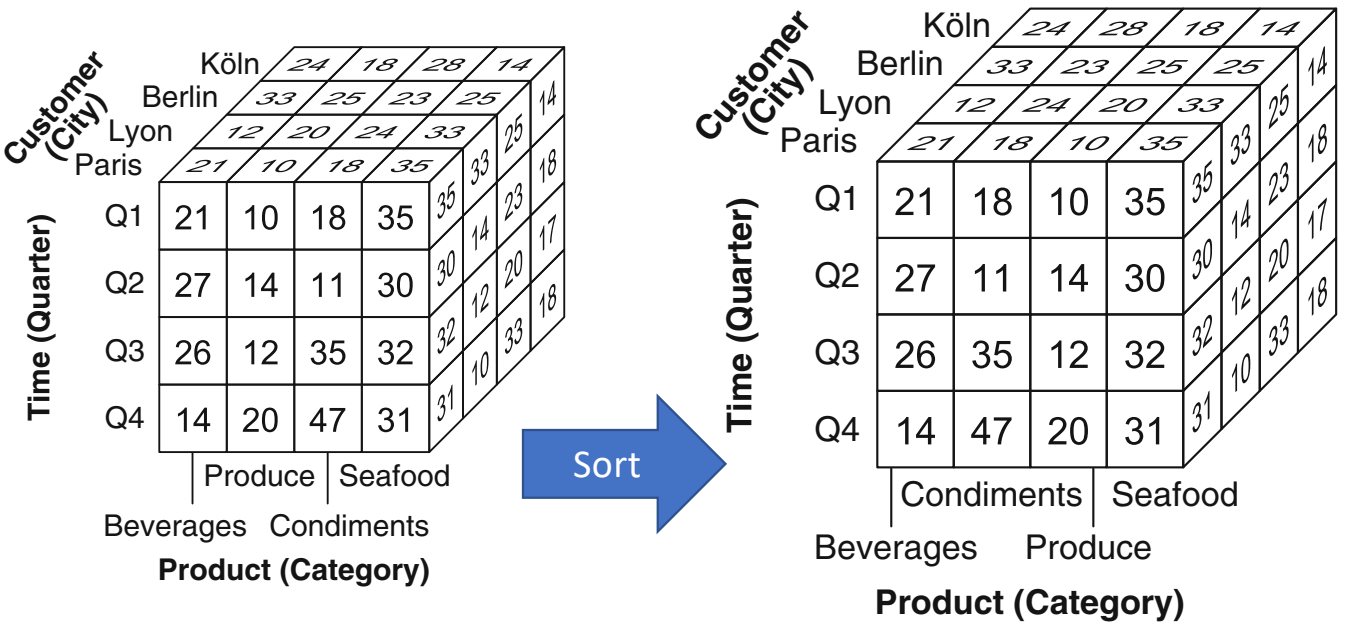
Our user now wants to explore alternative visualizations of the cube to better understand the data contained in it.

For this, she sorts the products by name

Then, she wants to see the cube with the Time dimension on the x-axis

Therefore, she takes the original cube and rotates the axes of the cube without changing granularities

Sort

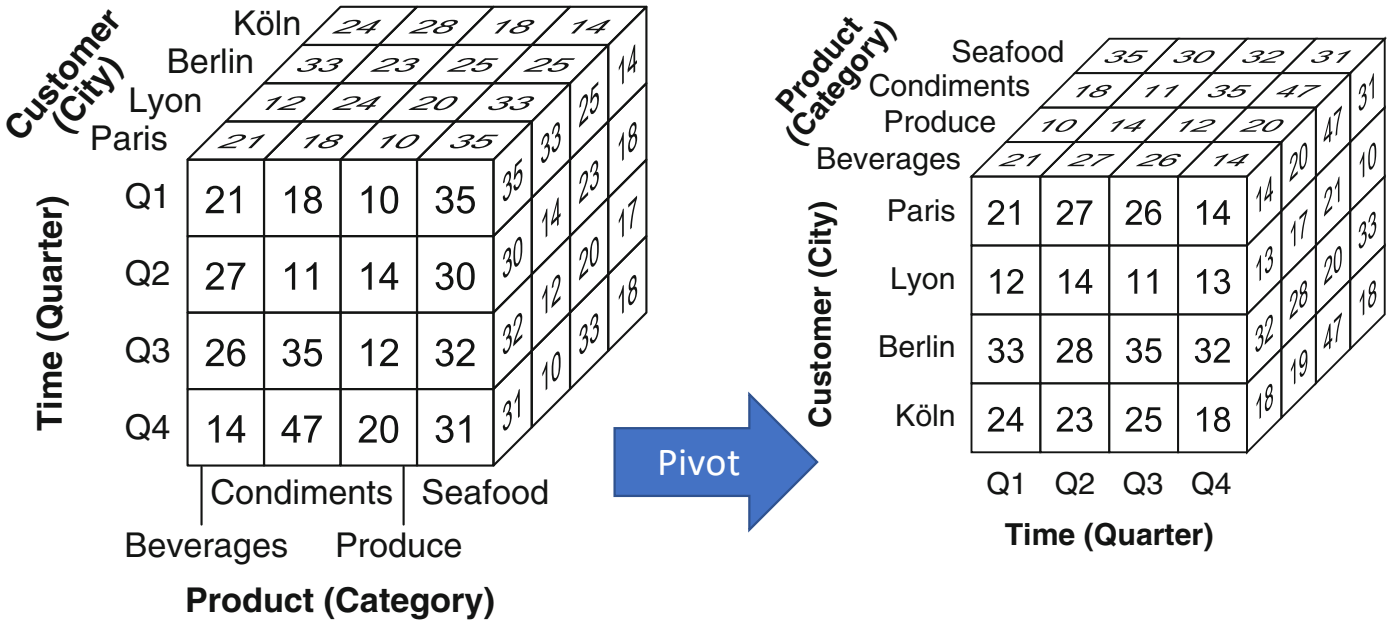


- The sort operation returns a cube where the members of a dimension have been sorted.
- The syntax of the operation is as follows:

  SORT(CubeName, Dimension, (Expression [{ASC | DESC | BASC | BDESC}])*)

- where the members of Dimension are sorted according to the value of Expression either in ascending or descending order. In the case of ASC or DESC, members are sorted within their parent, whereas in the case of BASC or BDESC, the sorting is performed across all members (i.e., irrespective of the hierarchies). The ASC is the default option.
- For example, the following expression **SORT(Sales, Product, ProductName)** sorts the members of the Product dimension in ascending order of their name

Pivot



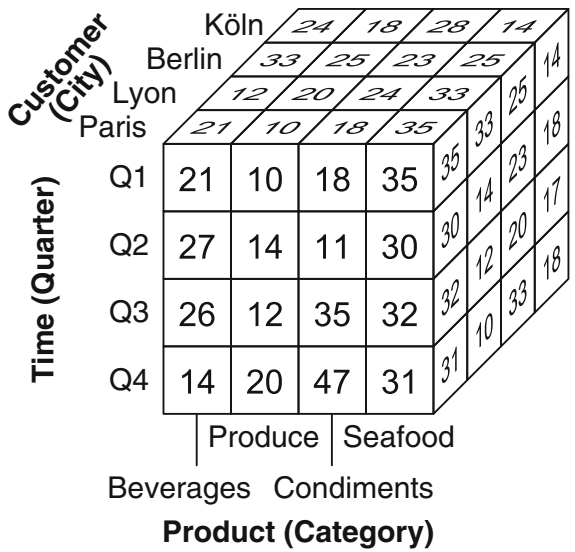
- The pivot (or rotate) operation rotates the axes of a cube to provide an alternative presentation of the data. The syntax of the operation is as follows:

  PIVOT(CubeName, (Dimension → Axis)*)

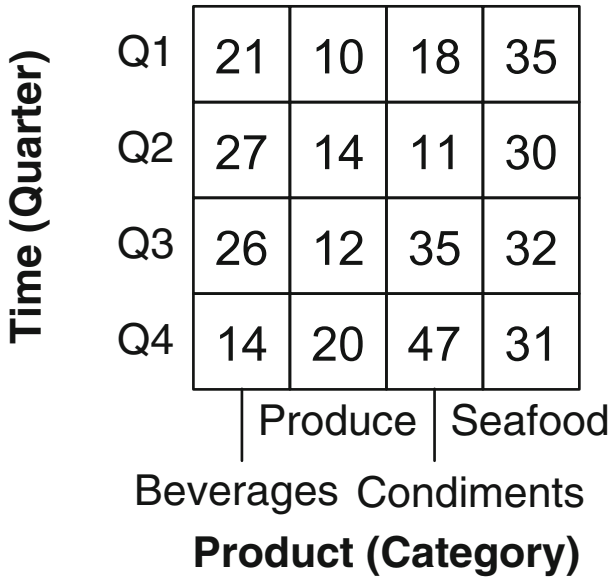
- where the axes are specified as {X, Y, Z, X1, Y1, Z1, . . . }.
- Thus, the example above is expressed by:

  PIVOT(Sales, Time → X, Customer → Y, Product → Z)

Example



Slice

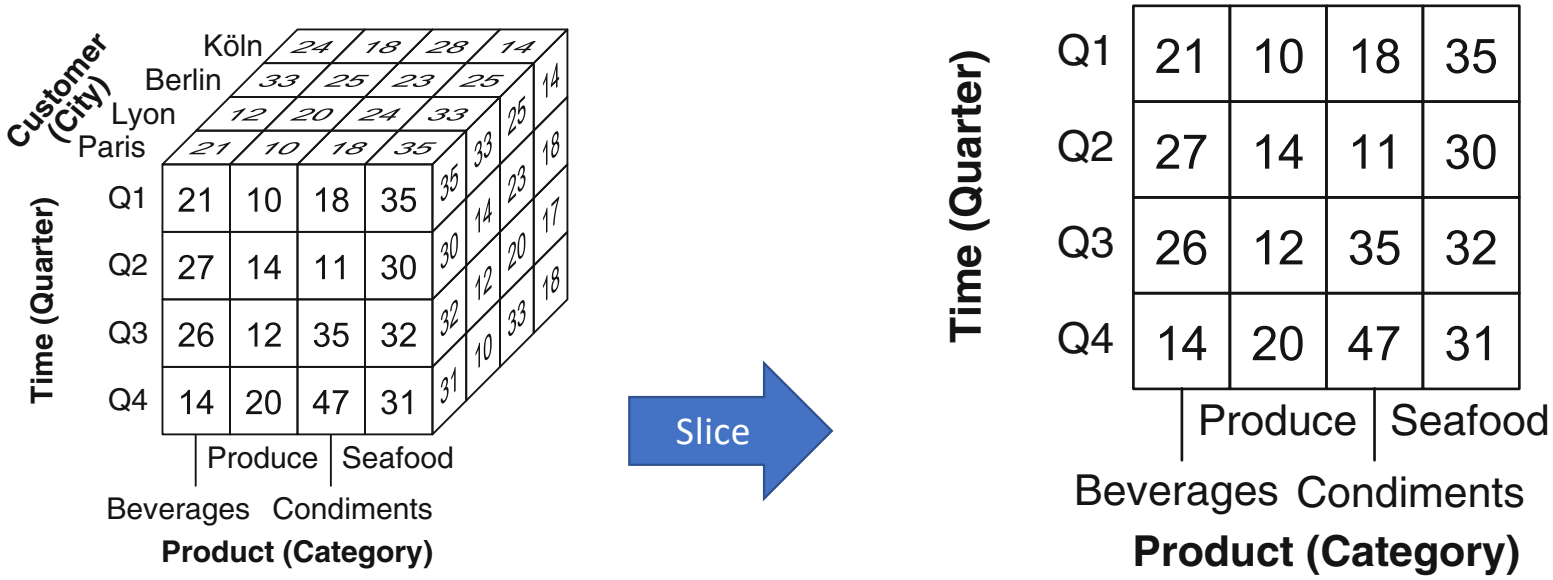


The user then wants to visualize the data only for Paris.

For this, she applies a slice operation

She obtained a two- dimensional matrix, where each column represents the evolution of the sales quantity by category and quarte

Slice



- The slice operation removes a dimension in a cube, that is, a cube of n − 1 dimensions is obtained from a cube of n dimensions. The syntax of this operation is:

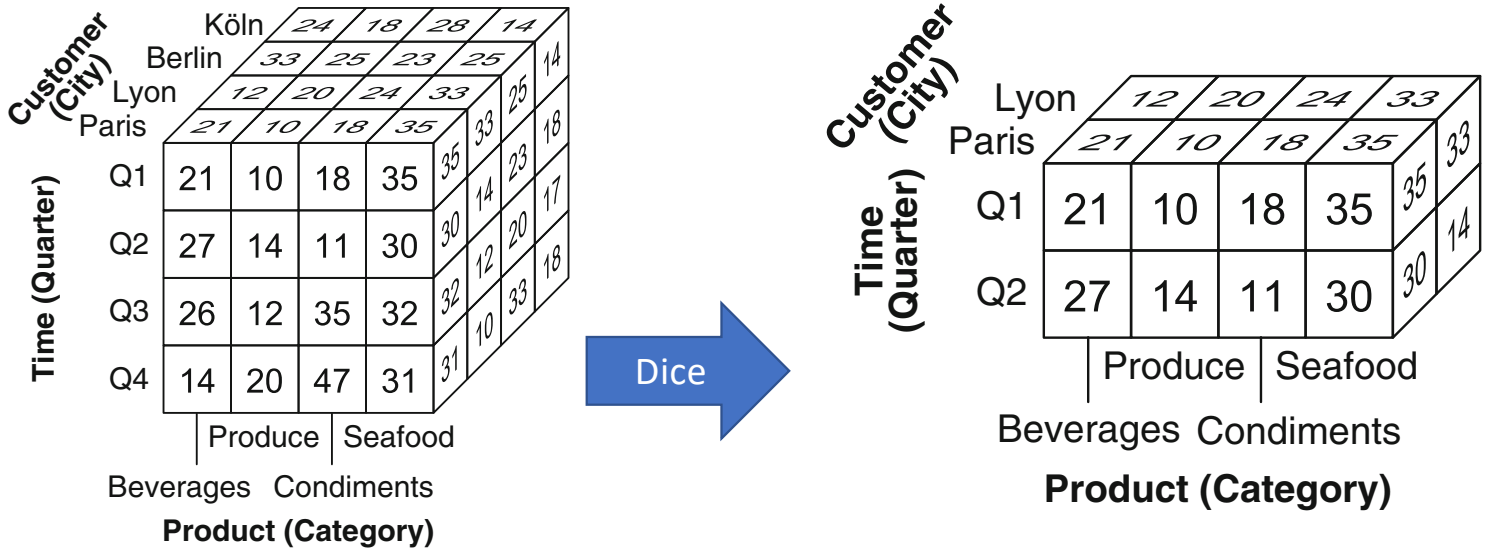
  SLICE(CubeName, Dimension, Level = Value)

- where the Dimension will be dropped by fixing a single Value in the Level. The other dimensions remain unchanged. The example illustrated in the figure is expressed by:

  SLICE(Sales, Customer, City = 'Paris')

- The slice operation assumes that the granularity of the cube is at the specified level of the dimension (in the example above, at the city level).

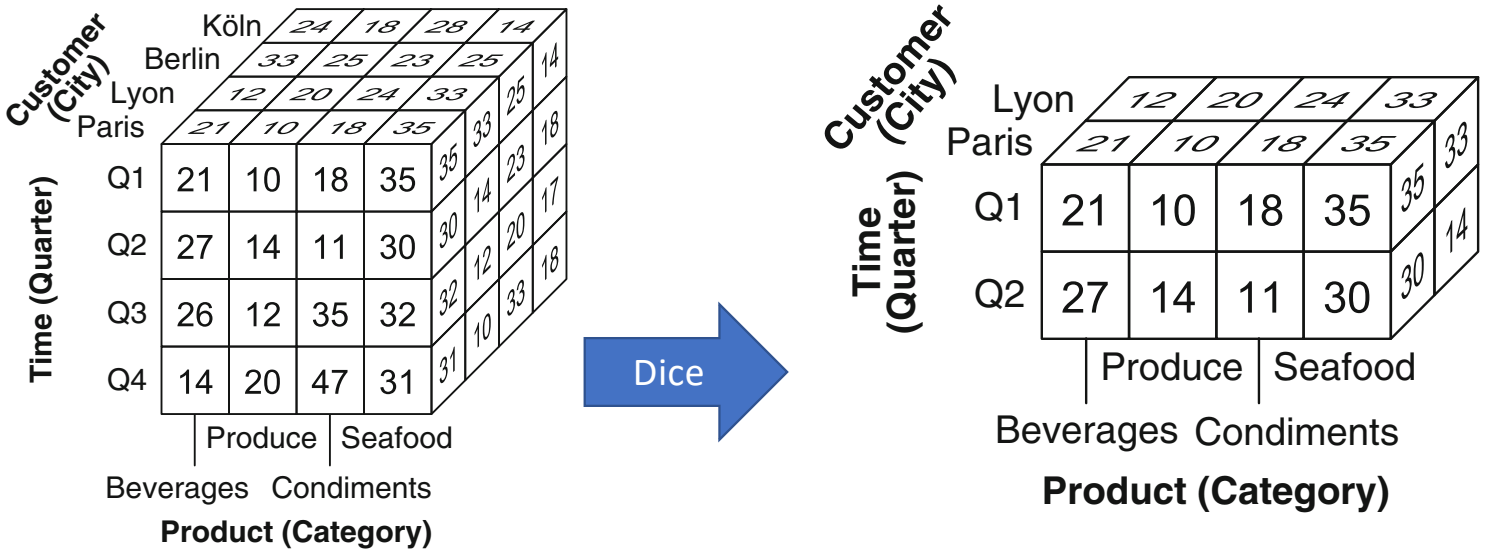
Example



Our user goes back to the original cube and builds a subcube, with the same dimensionality, but only containing sales figures for the first two quarters and for the cities Lyon and Paris.

This is done with a dice operation

Dice

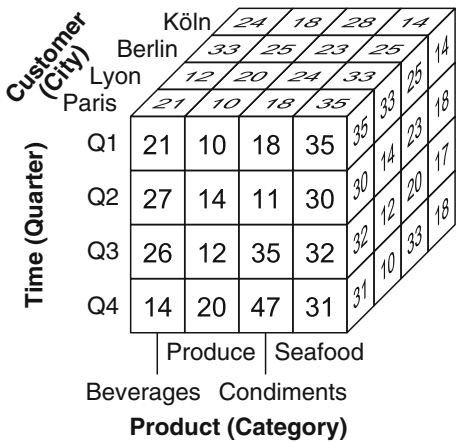


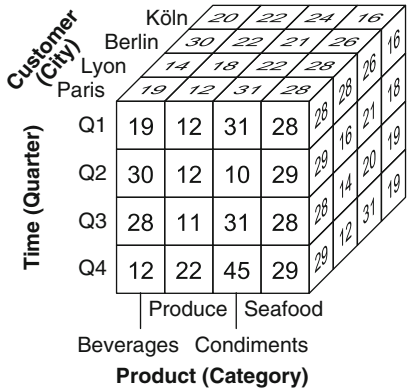
- The dice operation keeps the cells in a cube that satisfy a Boolean condition Φ. The syntax for this operation is

  DICE(CubeName, Φ)

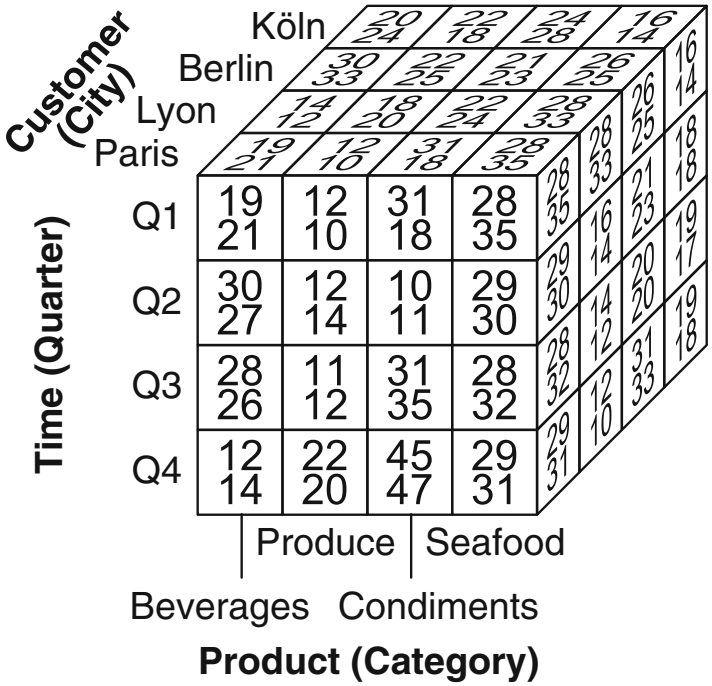
- where Φ is a Boolean condition over dimension levels, attributes, and measures. The DICE operation is analogous to the relational algebra selection σΦ(R), where the argument is a cube instead of a relation. The example illustrated in the figure is expressed by:

DICE(Sales, (Customer.City = 'Paris' OR Customer.City = 'Lyon') AND (Time.Quarter = 'Q1' OR Time.Quarter = 'Q2'))





Drill-Across



Our user now wants to compare the sales quantities in 2012 with those in 2011.

She wants to have the measures in the two cubes consolidated in a single one.

She uses the drill-across operation that, given two cubes, builds a new one with the measures of both in each cell

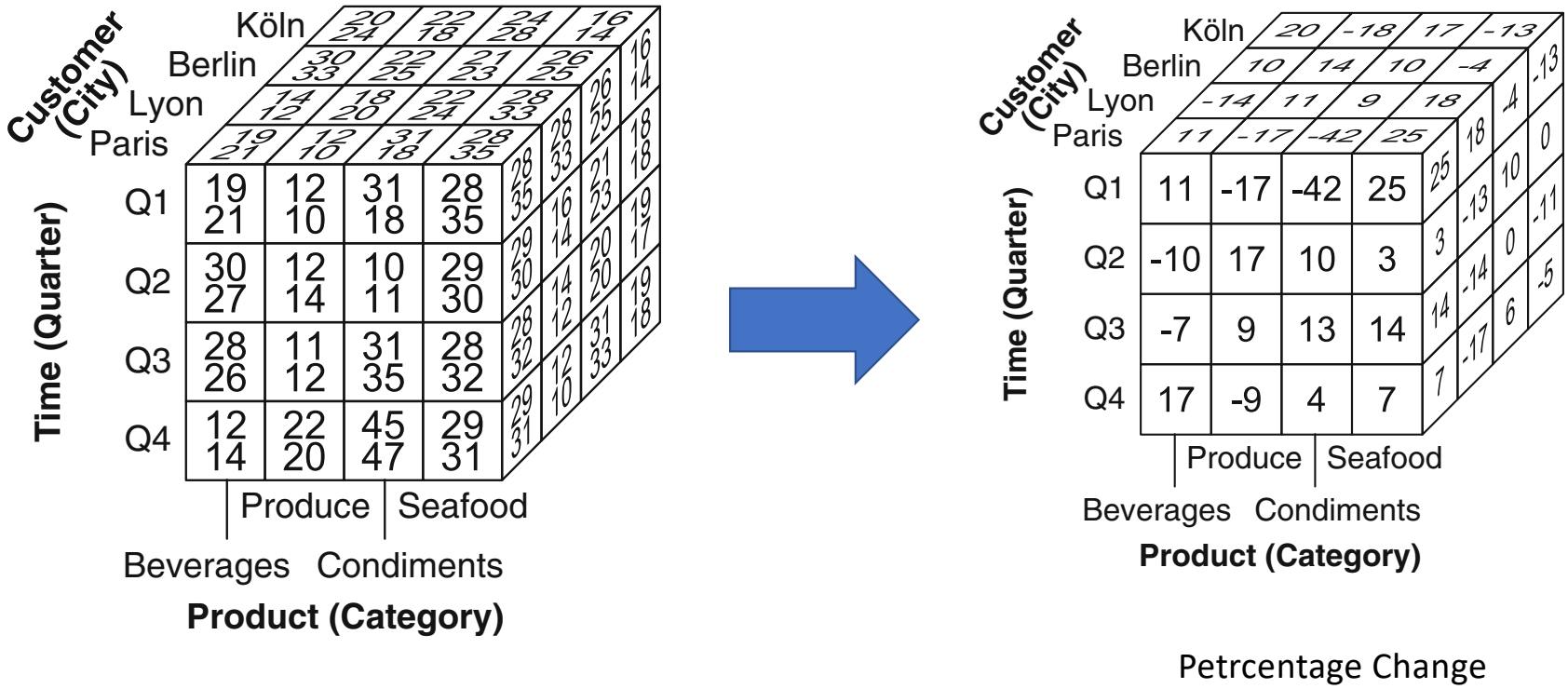
Drill-Cross

- The drill-across operation combines cells from two data cubes that have the same schema and instances. The syntax of the operation is:

  DRILLACROSS(CubeName1, CubeName2, [Condition])

- The DRILLACROSS operation is analogous to a full outer join in the relational algebra. If the condition is not stated, it corresponds to an outer equijoin.
- For example:

  Sales2011-2012 ← DRILLACROSS(Sales2011, Sales2012)



The user now wants to compute the percentage change of sales between the 2 years.

For this, she takes the cube resulting from the drill-across operation, and applies to it the add measure operation, which computes a new value for each cell from the values in the original cube.

Add

- The add measure operation adds new measures to the cube computed from other measures or dimensions. The syntax for this operation is as follows:

  ADDMEASURE(CubeName, (NewMeasure = Expression, [AggFct])*)

- where Expression is a formula over measures and dimension members and AggFct is the default aggregation function for the measure, SUM being the default.

- In the example in the previous slide, we have:

ADDMEASURE(Sales2011-2012, PercentageChange = (Quantity2011-Quantity2012)/Quantity2011)

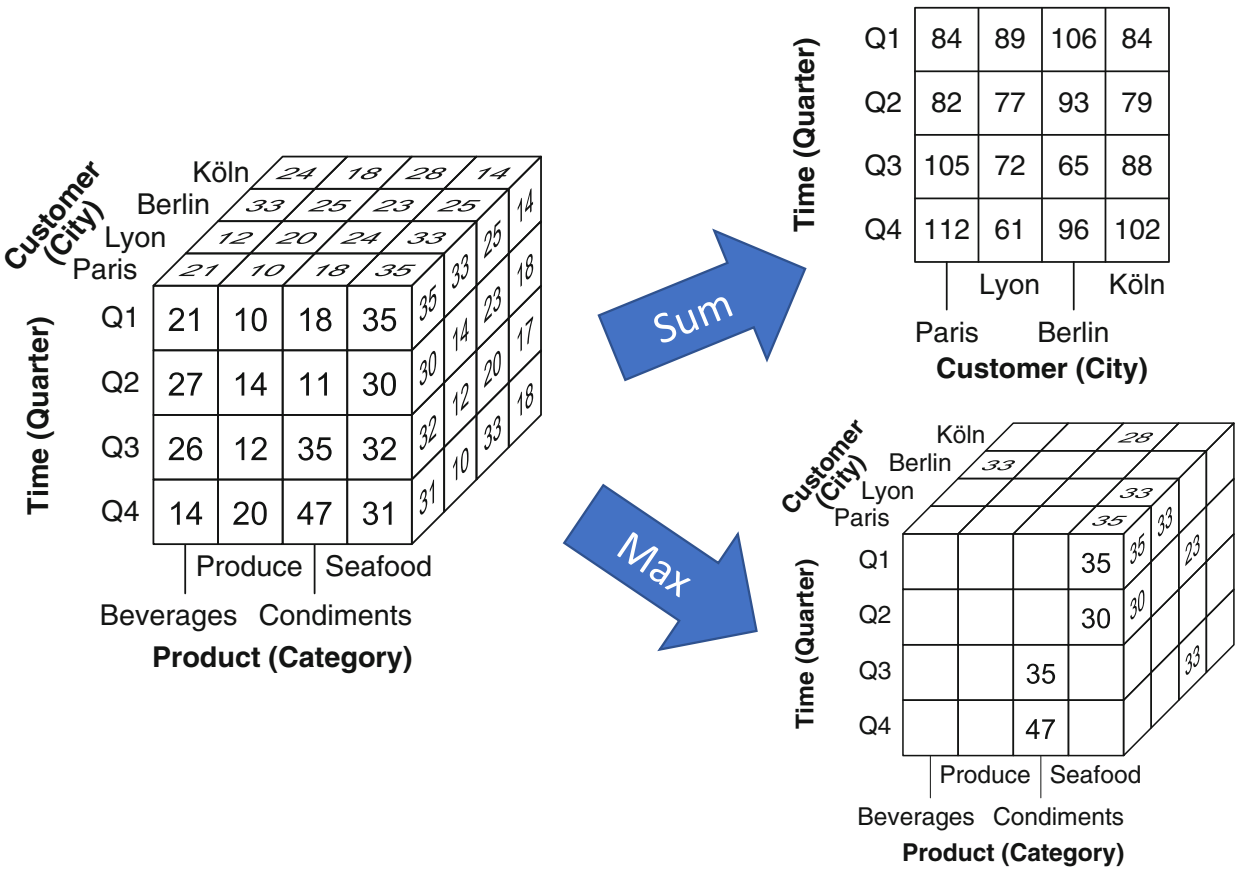
Drop

- The drop measure operation removes one or several measures from a cube. The syntax is as follows:

DROPMEASURE(CubeName, Measure*)

- For example, given the result of the add measure in the previous slide, we can use the following operation to keep only the percentage

DROPMEASURE(Sales2011-2012, Quantity2011, Quantity2012)

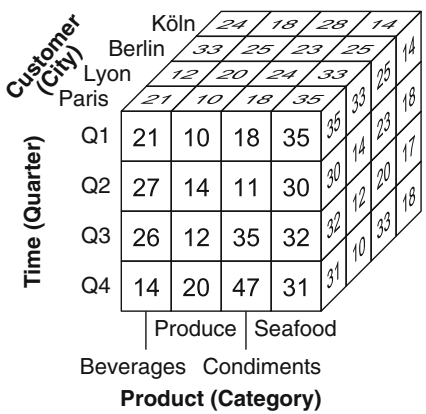


The user wants to aggregate data in various ways

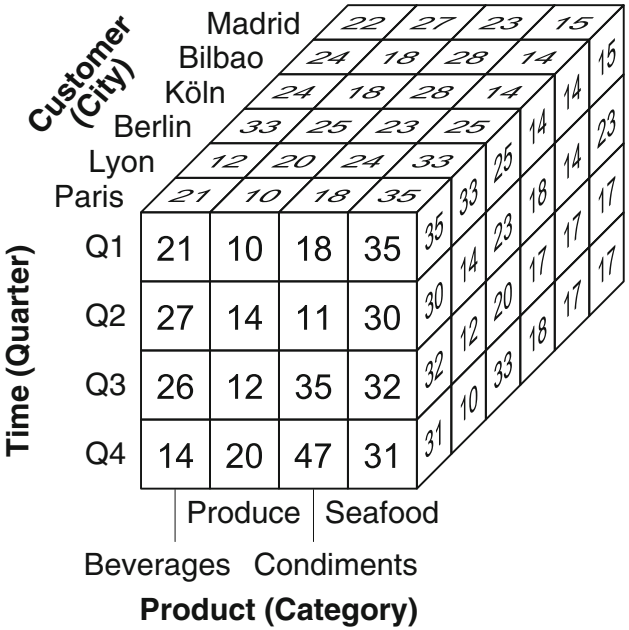
Given the original cube, she first wants to compute to total sales by quarter and city.

Then, the user wants to obtain the maximum sales by quarter and city, and for this

She uses the max operation



With a cube for Spain



The user wants to add to the original cube data from Spain, which are contained in another cub

She obtains this by performing a union of the two cubes

Union and Difference

- The union operation merges two cubes that have the same schema but disjoint instances.
- For example, if CubeSpain is a cube having the same schema as our original cube but containing only the sales to Spanish customers, then we can apply:

  UNION(Sales, SalesSpain)

- The difference operation removes the cells in a cube that exist in another one. The two cubes must have the same schema.

  DIFFERENCE(Sales, TopTwoSales)

Logical Modeling of Data Warehouses

Three approaches depending on how the data cube is stored:

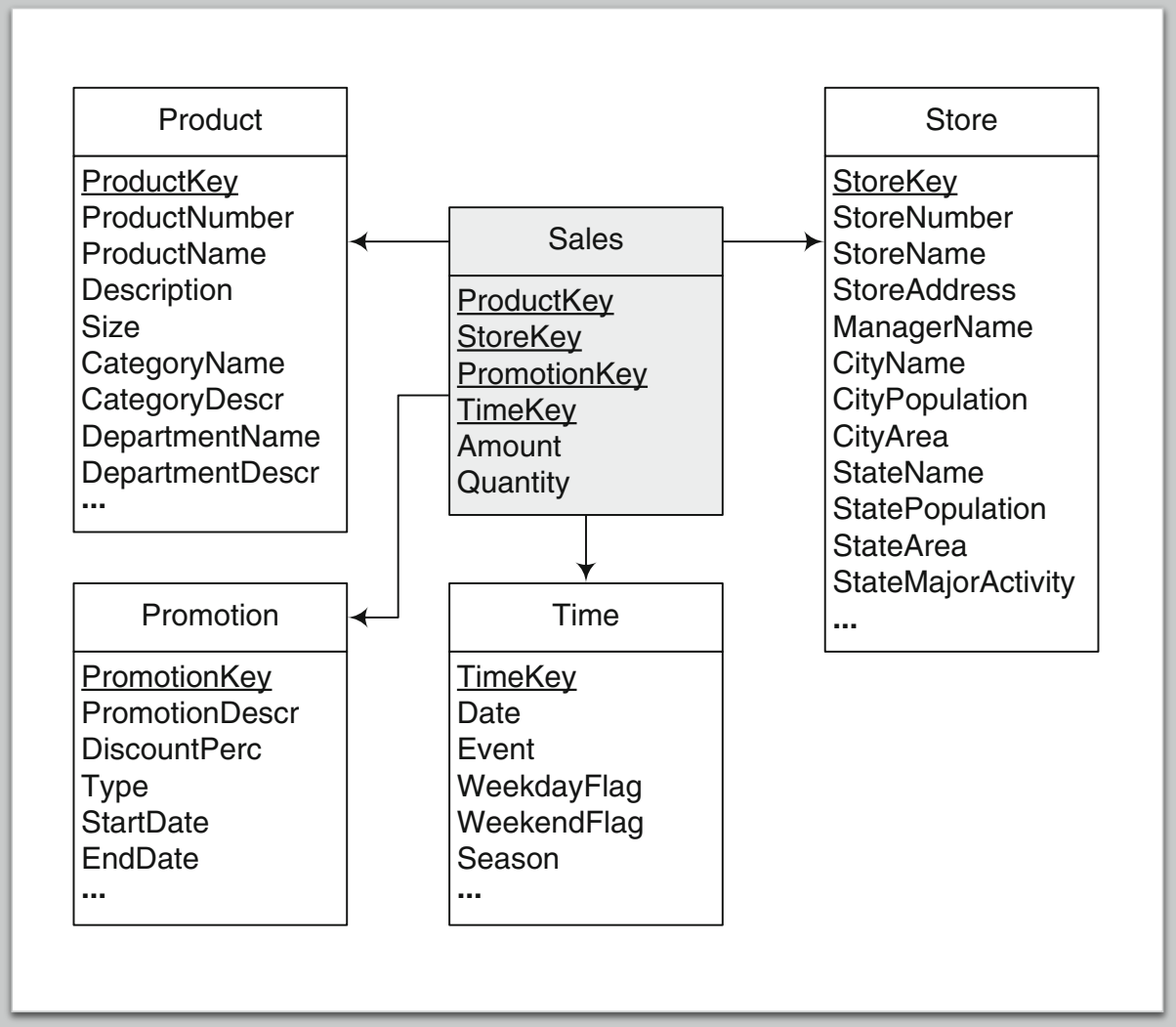
- Relational OLAP (**ROLAP**): stores data in relational databases and supports extensions to SQL.
  - Query language: Extension SQL/OLAP
  - Most relational databases implement such extension
- Multidimensional OLAP (**MOLAP**): stores data in specialized multidimensional data structures and implements the OLAP operations over those data structures.
  - Query language: MDX (Multidimensional Expressions)
  - System: example Mondrian (https://mondrian.pentaho.com/)
- Hybrid OLAP (**HOLAP**), which combines both approaches.

We will focus in what follows on ROLAP mainly because it has a large use base and is supported by conventional relational databases…

Relational Data Warehouse Design: Star Schema

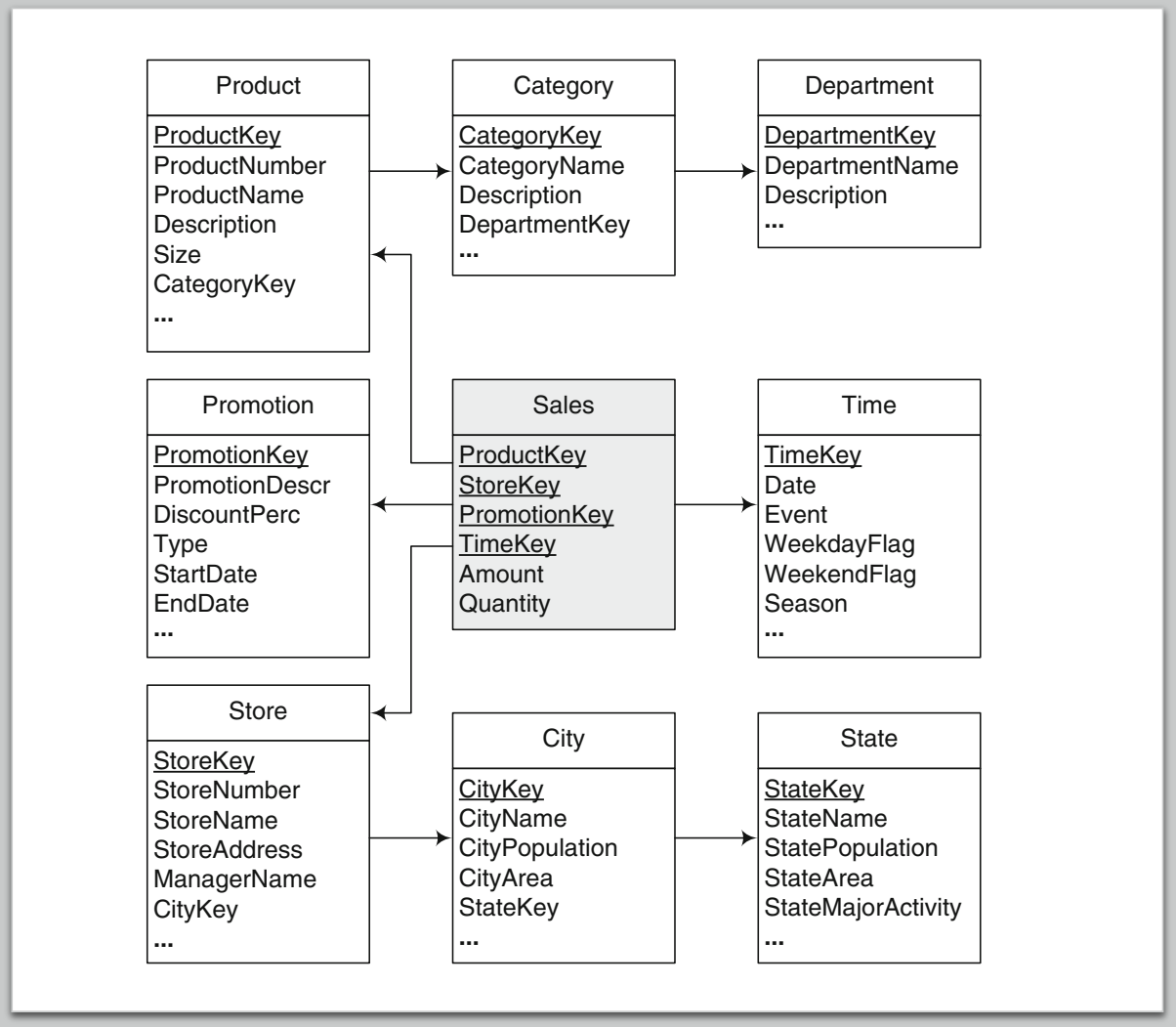
One possible relational representation of the multidimensional model is based on the star schema

- one central fact table, and
- a set of dimension tables, one for each dimension.



Relational Data Warehouse Design: Snowflake Schema

- A snowflake schema avoids the redundancy of star schemas by normalizing the dimension tables.
- Therefore, a dimension is represented by several tables related by referential integrity constraints.



Relational Datawarehouse Design

- Normalized tables are easy to maintain and optimize storage space.
- However, performance is affected since more joins need to be performed when executing queries that require hierarchies to be traversed.
- Example query “Total sales by category”

```
SELECT    CategoryName, SUM(Amount)
FROM      Product P, Sales S
WHERE     P.ProductKey = S.ProductKey
GROUP BY CategoryName
```

Star Schema

```
SELECT    CategoryName, SUM(Amount)
FROM      Product P, Category C, Sales S
WHERE     P.ProductKey = S.ProductKey AND P.CategoryKey = C.CategoryKey
GROUP BY CategoryName
```

Snowflake Schema

SQL/OLAP Operations

- We showed how the data cube, a multidimensional structure, can be represented in the relational model.
- We now show how to implement the OLAP operations in SQL using the extension called SQL/OLAP.
- A relational database is not the best data structure to hold data that is, in nature, multidimensional.
- Consider a simple cube Sales, with two dimensions, Product and Customer, and a measure, SalesAmount

	c1	c2	c3	Total
p1	100	105	100	305
p2	70	60	40	170
p3	30	40	50	120
Total	200	205	190	595

Data cube

ProductKey	CustomerKey	SalesAmount
p1	c1	100
p1	c2	105
p1	c3	100
p2	c1	70
p2	c2	60
p2	c3	40
p3	c1	30
p3	c2	40
p3	c3	50

Fact table

SQL/OLAP Operations

- How can we compute all aggregations along the two dimensions of category and product using SQL?
- A possible way to compute this in SQL is to use the NULL value as follows:

```
SELECT    ProductKey, CustomerKey, SalesAmount
FROM      Sales
    UNION
SELECT    ProductKey, NULL, SUM(SalesAmount)
FROM      Sales
GROUP BY ProductKey
    UNION
SELECT    NULL, CustomerKey, SUM(SalesAmount)
FROM      Sales
GROUP BY CustomerKey
    UNION
SELECT    NULL, NULL, SUM(SalesAmount)
FROM      Sales
```

	c1	c2	c3	Total
p1	100	105	100	305
p2	70	60	40	170
p3	30	40	50	120
Total	200	205	190	595

Data cube

ProductKey	CustomerKey	SalesAmount
p1	c1	100
p1	c2	105
p1	c3	100
p2	c1	70
p2	c2	60
p2	c3	40
p3	c1	30
p3	c2	40
p3	c3	50

Fact table

SQL/OLAP Operations

- How can we compute all aggregations along the two dimensions of category and product using SQL?
- A possible way to compute this in SQL is to use the NULL value as follows:

```
SELECT   ProductKey, CustomerKey, SalesAmount
FROM     Sales
    UNION
SELECT   ProductKey, NULL, SUM(SalesAmount)
FROM     Sales
GROUP BY ProductKey
    UNION
SELECT   NULL, CustomerKey, SUM(SalesAmount)
FROM     Sales
GROUP BY CustomerKey
    UNION
SELECT   NULL, NULL, SUM(SalesAmount)
FROM     Sales
```

	c1	c2	c3	Total
p1	100	105	100	305
p2	70	60	40	170
p3	30	40	50	120
Total	200	205	190	595

Data cube

ProductKey	CustomerKey	SalesAmount
p1	c1	100
p2	c1	70
p3	c1	30
NULL	c1	200
p1	c2	105
p2	c2	60
p3	c2	40
NULL	c2	205
p1	c3	100
p2	c3	40
p3	c3	50
NULL	c3	190
p1	NULL	305
p2	NULL	170
p3	NULL	120
NULL	NULL	595

Query result

SQL/OLAP Operations

- Computing a cube with n dimensions in the way described in the previous slide is not very efficient.
- For this reason, SQL/OLAP extends the GROUP BY clause with the ROLLUP and CUBE operators.
- ROLLUP computes group subtotals in the order given by a list of attributes.

```
SELECT    ProductKey, CustomerKey, SUM(SalesAmount)
FROM      Sales
GROUP BY ROLLUP(ProductKey, CustomerKey)
```

ProductKey	CustomerKey	SalesAmount
p1	c1	100
p1	c2	105
p1	c3	100
p1	NULL	305
p2	c1	70
p2	c2	60
p2	c3	40
p2	NULL	170
p3	c1	30
p3	c2	40
p3	c3	50
p3	NULL	120
NULL	NULL	595

Query result

SQL/OLAP Operations

- Computing a cube with n dimensions in the way described in the previous slide is not very efficient.
- For this reason, SQL/OLAP extends the GROUP BY clause with the ROLLUP and CUBE operators.
- CUBE computes all totals of such a list.

```
SELECT    ProductKey, CustomerKey, SUM(SalesAmount)
FROM      Sales
GROUP BY CUBE(ProductKey, CustomerKey)
```

ProductKey	CustomerKey	SalesAmount
p1	c1	100
p2	c1	70
p3	c1	30
NULL	c1	200
p1	c2	105
p2	c2	60
p3	c2	40
NULL	c2	205
p1	c3	100
p2	c3	40
p3	c3	50
NULL	c3	190
NULL	NULL	595
p1	NULL	305
p2	NULL	170
p3	NULL	120

Query result

Exercise 1: Star Schema

Consider A data warehouse of a telephone provider consists of five dimensions: caller customer, callee customer, time, call type, and call program and three measures: number of calls, duration, and amount.

- Draw a star schema diagram for the data warehouse.

Exercise 1: SQL/OLAP Queries

Write in SQL the following queries:

1. Total amount collected by each call program in 2012.
2. Total duration of calls made by customers from Brussels in 2012.
3. Total number of weekend calls made by customers from Brussels to customers in Antwerp in 2012.
4. Total duration of international calls started by customers in Belgium in 2012.
5. Total amount collected from customers in Brussels who are enrolled in the corporate program in 2012.